



Hanzo AI Chain (AIVM)

The AI Virtual Machine, Native AI Coin, and Open Compute Markets on Lux Quasar 3.0

Zach Kelling

Hanzo AI

Spec freeze: 15 December 2025 | Final review: 20 December 2025 | Activation: 25 December 2025

Abstract

We present the Hanzo *AI Chain*, the AI Virtual Machine (AIVM) and the native economic substrate of the Hanzo Network. The AI Chain is a public re-framing of the Lux A-Chain (LP-134, the unified attestation chain): the same underlying ledger, certified by the same Quasar 3.0 three-lane post-quantum consensus (LP-105), executed on the same QuasarGPU substrate (LP-132), and scheduled by the same lane-based Quasar Software Transactional Memory model (LP-010). Where Lux describes A-Chain as a settlement and attestation layer for chains, Hanzo describes the AI Chain as the open AI protocol: a single chain that mints **AI**, the native coin of Hanzo Network, and that hosts on-chain registries for Hanzo Models (HMMs), agents, and three composable compute markets (inference, training, GPU pool). Each compute job is bound to a Trusted Execution Environment quote anchored on A-Chain; each inference receipt is a Quasar-certified proof of correct execution; each model is a content-addressed root with on-chain provenance, license, and benchmark commitments. Confidential inference is delegated to F-Chain (LP-013, LP-067 confidential ERC-20 patterns extended to model queries), key custody to M-Chain, and bridging to B-Chain (LP-134). The AI Chain activates on 25 December 2025 alongside Lux Quasar 3.0; the spec described here is frozen as of 15 December 2025.

Contents

1	Introduction	5
1.1	Why an AI Chain	5
1.2	Decentralized AI as a Three-Sided Market	5
1.3	Position vs. Centralized AI Labs	5
1.4	Contributions and Outline	6
2	Architecture	6
2.1	AI Chain = A-Chain, Reframed	6
2.2	Settlement: Lux Quasar 3.0 (LP-105)	6
2.3	Execution: QuasarGPU (LP-132)	7
2.4	Scheduling: QuasarSTM Lanes (LP-010)	7
2.5	The AIVM Opcode Surface	7
2.6	Block Time and Throughput	8
3	The AI Native Coin	8
3.1	Token vs. Coin	8
3.2	Supply Schedule	8
3.3	Block Reward	8
3.4	Fee Burn	9
3.5	Staking and Validator Yield	9
3.6	Worked Example: Cost of an Inference Call	9
4	Hanzo Models (HMMs)	10
4.1	What Lives On Chain	10
4.2	Versioning and Provenance	10
4.3	License Enforcement	11
4.4	Price Function	11
4.5	Benchmark Commitments and Challenges	11
4.6	Native Hanzo Models at Activation	11

5	Compute Markets	12
5.1	Inference Market	12
5.2	Training Market	12
5.3	GPU Pool Market	13
5.4	TEE Attestation Across All Three Markets	13
5.5	Spot vs. Reserved Capacity	13
5.6	Why Markets, Not a Single Centralized Scheduler	14
6	Agent Registry	14
6.1	Agents as First-Class Citizens	14
6.2	Persistent Identity	14
6.3	Capability Declaration	14
6.4	Reputation	15
6.5	Cross-Chain Authorization	15
6.6	Privacy of Agent Memory	15
6.7	Worked Example: A Research Agent	15
7	Cross-Chain Integration	16
7.1	C-Chain (EVM)	16
7.2	F-Chain (FHE Confidential Inference)	16
7.3	M-Chain (Threshold Custody)	16
7.4	B-Chain (Bridges)	17
7.5	P-Chain, Q-Chain, Z-Chain (Consensus Substrate)	17
7.6	Topology Summary	17
8	Privacy	17
8.1	Threat Model	17
8.2	Public Inference	17
8.3	Confidential Inference (F-Chain Delegation)	18
8.4	Threshold Custody for Model Weights	18
8.5	Receipt Privacy	18
8.6	Privacy Layering Summary	18
8.7	What Is Not Hidden	18
9	Security	19
9.1	Three-Lane Post-Quantum Safety, Inherited	19
9.2	What an AI Chain Adversary Must Break	19
9.3	Operator Honesty	19
9.4	Sybil Resistance	20
9.5	Liveness	20
9.6	Replay and Cross-Chain Attacks	20
9.7	Audit Surface	20
10	Deployment	20
10.1	Activation Boundary	20
10.2	Pre-Activation Genesis State	21
10.3	Operator Onboarding	21
10.4	Caller Onboarding	21
10.5	Activation Roadmap	21
10.6	Post-Activation: First 90 Days	21
10.7	Where Things Run	22

1 Introduction

1.1 Why an AI Chain

The economics of artificial intelligence are concentrating. A small number of hyperscalers control the GPUs that train frontier models, the model weights themselves, the APIs through which those models are sold, and the price at which inference is offered. This concentration produces three structural problems for everyone who is not a hyperscaler. First, *access*: researchers, agents, and applications cannot build on weights they cannot inspect or fine-tune. Second, *pricing*: there is no spot market for inference, only published price lists. Third, *provenance*: model outputs arrive without a verifiable record of which model produced them, on what hardware, at what version. A user must trust the API.

The Hanzo AI Chain replaces those three trust assumptions with verifiable on-chain primitives. Models are content-addressed and versioned on chain. Inference is sold through an open market in which any operator may list capacity and any caller may submit jobs. Each job receipt is a Quasar certificate (LP-105) that attests, in a single 192-byte SNARK plus a 32-byte public-input hash, both that the consensus quorum signed off on the result and that the executing GPU produced a TEE quote pinning the exact model commitment, hardware identity, and runtime version.

1.2 Decentralized AI as a Three-Sided Market

The AI Chain organizes the global compute market as three interlocking sides:

- **Model owners** publish weights, license terms, evaluation results, and an inference price function. The chain stores commitments, not weights; large weight blobs live in content-addressed storage and are referenced by their root hash.
- **Compute operators** stake AI to register GPU capacity, produce TEE attestations on every job, and earn a share of the per-token fee plus a block reward.
- **Callers** (humans, applications, agents) submit jobs, pay in AI, and receive a Quasar-signed receipt that proves which model produced which output on which hardware.

These three sides are mediated by a fourth component, the *agent registry*, which gives autonomous Hanzo Agents (`~/work/hanzo/agents`) a persistent on-chain identity, an explicit capability declaration, and a reputation that follows them across jobs.

1.3 Position vs. Centralized AI Labs

A centralized AI lab is, from a buyer's standpoint, an opaque oracle: input in, output out, no proof of anything in between. The AI Chain offers the opposite primitive. Every output is accompanied by:

1. A *model commitment* (§4) pinning the exact weights and architecture that produced the output;
2. A *TEE attestation quote* (LP-134) pinning the hardware identity and runtime version;
3. A *QuasarCert* (LP-105) anchoring both of the above into the canonical chain history with three independent post-quantum verification lanes.

This is the same security envelope the Lux network uses for value settlement, applied to AI computation. It is not an additional system; it is a re-framing of A-Chain as the AI Virtual Machine, with a native coin (AI), a registry of models (HMMs), and a market structure designed for compute rather than transfer.

1.4 Contributions and Outline

1. The AI Chain architecture (§2): A-Chain reframed as the AIVM, settled by Lux Quasar 3.0 (LP-105) and executed on QuasarGPU (LP-132) with QuasarSTM lane scheduling (LP-010).
2. The AI native coin (§3): block reward, fee burn, staking yields, and operator economics for the Hanzo Network.
3. The HMM registry (§4): on-chain provenance, versioning, license, benchmark, and price commitments for every Hanzo model.
4. Three composable compute markets (§5): per-token inference, training bounties, and GPU pool capacity.
5. The agent registry (§6): persistent identity, capability declaration, and on-chain reputation for autonomous agents.
6. Cross-chain integration (§7) with Lux C-Chain (EVM), F-Chain (FHE inference), M-Chain (threshold custody), and B-Chain (bridges) per LP-134.
7. Confidential inference (§8) and three-lane post-quantum security (§9).
8. Deployment plan (§10): activation 25 December 2025 on the same Quasar 3.0 boundary that activates LP-010, LP-105, LP-132, LP-134.

2 Architecture

2.1 AI Chain = A-Chain, Reframed

The Hanzo AI Chain is the Lux A-Chain (LP-134), the unified global attestation ledger, viewed as the AI Virtual Machine. There is exactly one chain; the term “AI Chain” is the public-facing identity of A-Chain when it is acting as the open AI protocol for the Hanzo Network. This avoids the proliferation of ledgers that has plagued every prior decentralized AI proposal: a single canonical chain hosts attestations for value-settlement chains (P/Q/Z lanes), and it hosts the AI registry, agent registry, compute markets, and AI coin economics described in this paper.

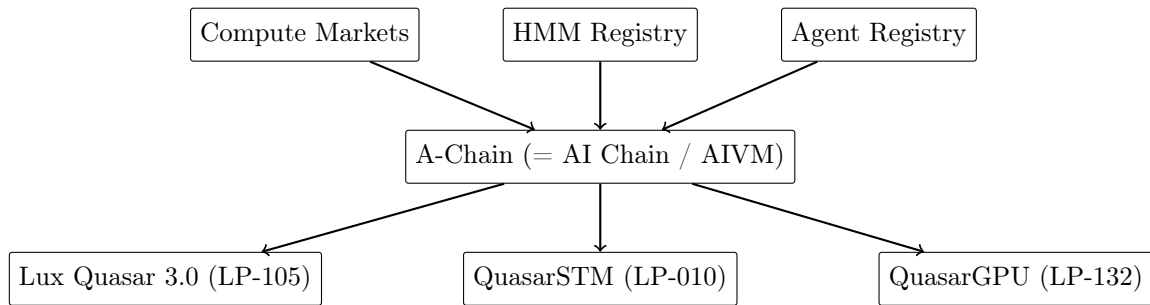


Figure 1: The AI Chain is the AIVM persona of A-Chain. It is settled by Quasar 3.0, executed by QuasarGPU, and scheduled by QuasarSTM. On top of that single ledger sit the HMM registry, the agent registry, and the compute markets.

2.2 Settlement: Lux Quasar 3.0 (LP-105)

Every block on the AI Chain is finalized by a single *QuasarCert* that aggregates three independent post-quantum verification lanes:

- **P-lane** — BLS12-381 aggregate signature (classical fast path);
- **Q-lane** — Ringtail two-round Ring-LWE threshold signature;
- **Z-lane** — a Z-Chain Groth16 SNARK rolling up N ML-DSA-65 validator signatures into a single 192-byte proof plus a 32-byte public-inputs hash.

Lane independence guarantees that an adversary must simultaneously break the discrete logarithm on BLS12-381, Module-LWE for Ringtail, and Module-LWE/MSIS for ML-DSA to forge a certificate. Breaking any one or any two lanes leaves the third intact. See LP-105 for the formal definition, the certificate-subject binding, and the security proofs.

For the AI Chain, this means every inference receipt and every market state transition inherits the same three-lane post-quantum envelope used to settle LUX value transfers. There is no separate, weaker security tier for “AI things.”

2.3 Execution: QuasarGPU (LP-132)

QuasarGPU is the GPU-resident execution substrate that verifies QuasarCerts and runs the AIVM. The same hardware that batches BLS aggregate verify ($\sim 875 \mu s$ for $n=100$), Ringtail batch verify ($O(1)$ amortized after DKG), and the single Groth16 pairing check ($200\text{--}500 \mu s$) also hosts the inference kernels invoked by AIVM opcodes. Co-location is deliberate: a GPU that has already loaded a Hanzo model into HBM can service the next inference job without paging weights from main memory, and it can produce the TEE quote that anchors the result without leaving the trusted boundary.

QuasarGPU exposes a small set of primitives the AIVM relies on:

- **ATTEST(quote)** — produce a vendor-rooted attestation quote (Intel TDX, AMD SEV-SNP, NVIDIA H100/H200/B100 confidential compute, ARM CCA) covering the loaded model commitment and runtime version;
- **INFER(model_root, input)** — run forward pass under the attested boundary, returning (output, quote);
- **VERIFY(quasarcert)** — verify a QuasarCert on GPU (used by light clients and by cross-chain bridges).

2.4 Scheduling: QuasarSTM Lanes (LP-010)

QuasarSTM (LP-010) is the lane-based software transactional memory model used by the Quasar pipeline. The AI Chain reuses the same lane abstraction: independent compute jobs that do not touch overlapping state run in parallel lanes; only state-touching transitions (operator stake updates, market settlement, model registry mutations) serialize. Because inference is read-mostly with respect to the chain (model commitments are read; only the receipt is written), the AI Chain achieves throughput limited by GPU capacity, not by chain serialization.

2.5 The AIVM Opcode Surface

The AIVM is a superset of the EVM with tensor-aware opcodes. Standard EVM opcodes execute unchanged. AI-specific operations are exposed as precompiles addressable from any EVM contract:

Address	Name	Purpose
0x0a01	HMM_LOOKUP	Resolve model_root $\rightarrow$ metadata
0x0a02	INFER_REQUEST	Submit inference job to market
0x0a03	INFER_VERIFY	Verify TEE quote + receipt
0x0a04	TRAIN_BOUNTY	Post training bounty
0x0a05	GPU_LEASE	Lease GPU pool capacity
0x0a06	AGENT_AUTH	Authorize agent action under cap
0x0a07	F_CALL	Cross-chain call to F-Chain (FHE)
0x0a08	M_CUSTODY	Cross-chain call to M-Chain (MPC)

Table 1: AIVM precompile addresses. EVM contracts may call any of these to interact with the AI Chain registries and markets without leaving the solidity programming model.

2.6 Block Time and Throughput

The AI Chain targets the same 500 ms block time and ≥ 100 K TPS-per-chain envelope as the rest of Quasar 3.0 (LP-105 §8). Inference throughput is bounded by aggregate operator GPU capacity, not by the chain itself; in practice each block can record thousands of inference receipts because each receipt is a fixed-size commitment, not a re-execution.

3 The AI Native Coin

3.1 Token vs. Coin

LUX is the native coin of the Lux primary network and pays for security and gas across all Lux chains. AI is the native coin of the Hanzo Network and is independent of LUX. The two coins are not the same asset, and neither is a wrapper of the other. AI is minted on the AI Chain, paid for compute on the AI Chain, burned proportionally to AI Chain fees, and staked by AI Chain operators. LUX is unaffected.

This separation is intentional. Security inflation (LUX) and AI compute inflation (AI) have different elasticities and different stakeholder sets; bundling them into one token would distort both. The architecture parallels how A-Chain (LP-134) separates the gas token from application-layer orchestration tokens such as AI and ZOO.

3.2 Supply Schedule

Parameter	Value
Genesis supply	1,000,000,000 AI
Block reward (initial)	4 AI / 500 ms block
Halving interval	every 4 years
Long-run terminal supply	2,000,000,000 AI
Activation block reward	2025-12-25 (block 0)

Table 2: AI supply parameters at genesis. The block reward halves every four years, asymptoting at the terminal supply.

The genesis supply is allocated as follows: 30% public sale and ecosystem liquidity, 25% operator and agent rewards reserve, 20% Hanzo Industries treasury (4-year linear vest), 15% research and core contributors (4-year linear vest, 1-year cliff), 10% Hanzo Foundation grants.

3.3 Block Reward

Each block emits the current epoch reward $R(t)$:

$$R(t) = R_0 \cdot 2^{-\lfloor t/T_h \rfloor} \quad (1)$$

where $R_0 = 4$ AI, $T_h = 4$ years, and t is wall time since activation (2025-12-25). The reward is split:

- 60% to compute operators in proportion to attested per-block work (§5);
- 30% to validators in proportion to QuasarCert participation weight (LP-105);
- 10% to the agent rewards pool, paid out as capability-weighted micro-grants to active agents (§6).

3.4 Fee Burn

Every paid action on the AI Chain carries a base fee in AI: inference calls (per token), training bounty escrow, GPU lease payments, registry mutations, and standard EVM gas. The base fee follows the same EIP-1559-style mechanism the EVM uses, but with a different burn split:

$$\text{base_fee_burned} = 0.7 \cdot \text{base_fee}, \quad \text{base_fee_to_treasury} = 0.3 \cdot \text{base_fee}. \quad (2)$$

The 70/30 burn ratio is calibrated such that, at sustained network usage above a threshold U^* (defined as 50% of theoretical block-fee capacity), net issuance becomes negative; AI becomes deflationary under heavy load. Below U^* , the 60/30/10 block reward dominates and net issuance is positive, subsidizing operator bootstrap.

3.5 Staking and Validator Yield

AI Chain validators stake AI to participate in Quasar 3.0 consensus on this chain. Stake mechanics inherit from LP-105 §3 (Quasar Validator Set): a validator must contribute to all three lanes (P, Q, Z) to earn rewards; lane-incompleteness slashes proportionally.

Operator staking is a separate, second-tier deposit governing right to list compute capacity (§5). Operator stake is locked against attestation fraud: a single proven false TEE quote slashes 100% of the operator’s stake, distributed to the verifier and to the burn address.

	Validator	Operator
Min stake	100,000 AI	10,000 AI
Lane participation reqd.	P + Q + Z	TEE quotes
Slashing for downtime	0.1%/epoch	—
Slashing for equivocation	100%	—
Slashing for false quote	—	100%
Target net yield (year 1)	7–9%	12–18%

Table 3: Validator vs. operator stake. Validators secure consensus; operators sell compute. Both stake AI; both can be slashed; yields differ because operator workloads carry capital and operating costs that validator-only operation does not.

3.6 Worked Example: Cost of an Inference Call

A caller submits an inference job to a 70B-parameter HMM at a posted price of $p = 0.000\,002$ AI per output token. The job produces 512 output tokens. Total payment $P = 0.001\,024$ AI, decomposed as:

- Operator share: $0.85 \cdot P = 8.704 \times 10^{-4}$ AI;

- Model owner royalty: $0.10 \cdot P = 1.024 \times 10^{-4}$ AI;
- Base fee: $0.05 \cdot P = 5.12 \times 10^{-5}$ AI, of which 3.584×10^{-5} is burned and 1.536×10^{-5} is routed to the treasury.

The operator’s GPU produced a TEE quote covering this job; the receipt is recorded in the next block’s QuasarCert. The caller can verify the receipt on any device with a single Groth16 pairing check.

4 Hanzo Models (HMMs)

4.1 What Lives On Chain

A Hanzo Model is a content-addressed, versioned bundle of weights, metadata, and license. The chain does *not* store weight blobs; those live in content-addressed storage (Hanzo object storage, **hanzo/storage**, or any compatible CAS) and are referenced by their root hash. The chain stores a small fixed-size record per model version that commits to everything a caller, an operator, or an auditor needs to verify a job:

Field	Definition
model_root	Merkle root over the weight tensors at version V . Content-addressed; pinning the root pins the weights bit-exactly.
benchmark_root	Merkle root over a canonical evaluation harness output (e.g. MMLU, HELM, internal evals). Submitted by the model owner; challengeable on chain.
license_root	Hash of the license document (commercial, research, or open under explicit terms), with a machine-readable summary.
inference_price	A price function $p(\cdot)$ in AI per output token (or per call, per byte for non-text models). May be a constant, a piece-wise function, or a reference to an on-chain pricing oracle.
training_history	Linked list of parent model_roots , each edge annotated with the training dataset commitment, the trainer identity, and the block height at which training completed.
owner	Address authorized to publish new versions and to update license / price; may be a multisig or a DAO.

Table 4: HMM registry record. Six fields per model version. The chain holds the commitments; weights and bench artifacts live in CAS.

4.2 Versioning and Provenance

Each new version of a model is a new record with a fresh **model_root** and a **training_history** edge pointing to the parent. This produces a directed acyclic graph rooted at the genesis of each model family. Anyone can walk the graph to see the full provenance: the original pretraining run, every fine-tune, every distillation, and the dataset commitment used at each step.

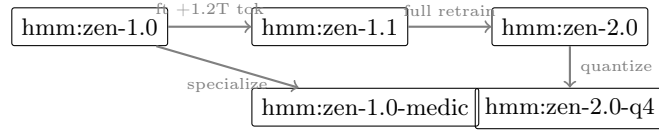


Figure 2: Provenance DAG. Each edge is a chain transaction recording the parent model_root, the training dataset commitment, and the trainer identity.

4.3 License Enforcement

The `license_root` commits to a license document with three machine-readable fields: *usage* (commercial / research / open), *redistribution* (allowed / restricted / prohibited), and *derivative* (allowed / restricted / prohibited). Compute operators are required to read and respect the license at job time; the `INFER_REQUEST` precompile (§2) refuses to schedule a job if the caller has not posted a matching license attestation.

4.4 Price Function

`inference_price` is a first-class on-chain object. The simplest form is a constant `uint256` in AI per output token. More expressive forms are supported:

- *Piecewise tiered*: a price ladder that lowers per-token cost above batch thresholds.
- *Time-of-day*: a function over block timestamp, used to clear demand at peak.
- *Oracle-backed*: a reference to an on-chain price oracle whose own update mechanism is visible on chain.

The model owner sets the function and may update it; updates are rate-limited (one update per epoch by default) to prevent surprise price moves between job submission and execution.

4.5 Benchmark Commitments and Challenges

A model owner publishes a `benchmark_root` when registering a version. Any address may post a *challenge*: a deposit-bonded claim that the published benchmark output does not match the actual output of the published `model_root` on the canonical eval harness. Challenges are resolved by a designated set of operators re-running the harness inside a TEE and signing the result. If the challenge succeeds, the challenger collects the model owner’s benchmark bond; if it fails, the challenger forfeits theirs. This matches the dispute pattern used elsewhere in Lux for attestation challenges (LP-134 §6).

4.6 Native Hanzo Models at Activation

At 25 December 2025 activation, the chain is genesis-seeded with the core Hanzo model family registered at version 1.0:

- `hmm:zen-text` (text-to-text foundation model);
- `hmm:zen-vision` (multimodal vision-language);
- `hmm:zen-audio` (speech and audio);
- `hmm:zen-code` (code-specialized);
- `hmm:zen-embed` (embedding model).

These are the production models served by the Hanzo LLM Gateway (`~/work/hanzo/llm`) and by Jin (`~/work/hanzo/jin`). Registration on chain is the single source of truth for which weights are canonical and at what price they are offered.

5 Compute Markets

The AI Chain hosts three composable compute markets. All three settle in AI, all three rely on TEE attestation for correctness, and all three share the operator stake and slashing mechanisms described in §3.

5.1 Inference Market

Listing. A compute operator with $\geq 10,000$ AI staked may list capacity for any registered HMM. A listing pins (`model_root`, `max_throughput`, `max_latency`, geographic region, supported precisions). Listings are indexed in an on-chain order book keyed by `model_root`.

Job submission. A caller submits an inference job by invoking the `INFER_REQUEST` precompile with (`model_root`, `input_root`, `max_payment`, `deadline`). The chain matches the job to a listing using the operator’s posted price function and the caller’s `max_payment` ceiling. The matched operator escrows the caller’s payment and starts the job.

Receipt. On completion the operator emits a receipt (`output_root`, `tee_quote`, `operator_sig`). The `tee_quote` covers the `model_root`, the runtime version, the `input_root`, and the `output_root`; this is what makes the result verifiable. The receipt is recorded in the block; payment is released from escrow; the operator’s stake is unaffected.

Throughput. Per-token pricing scales linearly with output length. Operators can batch many concurrent jobs on the same loaded model in HBM, so per-job fixed cost is small. Targets at activation: ≤ 200 ms first-token latency on 7B-class models, ≤ 2 s on 70B-class.

5.2 Training Market

Bounty. A model owner posts a bounty by invoking `TRAIN_BOUNTY` with (`parent_model_root`, `dataset_root`, `target_metric`, `target_threshold`, `payout`, `deadline`). The chain holds the payout in escrow.

Trainer. Any operator may attempt the bounty. The training run is conducted on the operator’s GPUs; the operator must produce a TEE attestation covering the training environment. On submission the operator emits a candidate (`new_model_root`, `training_log_root`, `tee_quote`).

Verification. Verifiers, drawn from the staked operator set by VRF, replay a sampled fraction α of the training steps inside their own TEEs and compare the resulting state. A successful replay releases the bounty; a failed replay slashes the candidate operator. This is the same verification-by-sampling pattern Hanzo ACI describes for general compute, specialized to training (`~/work/hanzo/papers/hanzo-aci`).

Settlement. On success the new model is registered as a child of the parent in the HMM provenance DAG (§4). The `training_log_root` is preserved in CAS, allowing third-party post-hoc audits.

5.3 GPU Pool Market

Capacity rental. Where the inference market sells per-token throughput against a specific model, the GPU pool market sells raw GPU-seconds against any workload. An operator lists pool capacity by GPU SKU (e.g. H100, H200, B100, B200, MI300X, TPUv5) and time window. A renter posts a job that includes a workload commitment and a TEE-verifiable runtime image.

Use cases. Pool rental is targeted at workloads that do not fit the inference or training markets: research experiments, custom kernels, fine-tuning runs that bring their own data and infra, benchmarking, ML systems research. The chain provides only the rental contract, the TEE attestation, and the settlement; the workload itself is opaque to the chain.

Pricing. Pool listings post a per-GPU-second price. Renters pay up front in AI into escrow; payment is released to the operator at the end of the window minus any documented downtime. Disputes are resolved by TEE log evidence.

5.4 TEE Attestation Across All Three Markets

All three markets rely on the A-Chain attestation primitive (LP-134). A market job is honest iff (a) the TEE quote validates against the correct vendor root, (b) the quoted measurement matches the agreed runtime image, and (c) the quoted input/output commitments match what the chain recorded. Any divergence is a slashable offense.

Algorithm 1 Honest-job check (executed by any verifier)

Require: receipt r , vendor root V , expected runtime image I

```
1:  $q \leftarrow r.\text{tee\_quote}$ 
2: if not VerifyQuote( $q, V$ ) then return SLASHOPERATOR
3: end if
4: if  $q.\text{measurement} \neq I.\text{measurement}$  then return SLASHOPERATOR
5: end if
6: if  $q.\text{inputs} \neq r.\text{input\_root}$  then return SLASHOPERATOR
7: end if
8: if  $q.\text{outputs} \neq r.\text{output\_root}$  then return SLASHOPERATOR
9: end if
10: return HONEST
```

5.5 Spot vs. Reserved Capacity

Operators may sell capacity in two pricing modes per market:

- *Spot*: dynamic price set per epoch, first-come-first-served, cancellable on either side with a small penalty. Targets latency- tolerant workloads at minimum cost.
- *Reserved*: a forward contract paid up front for a guaranteed capacity window. Targets predictable workloads (production inference, scheduled training).

The two modes share the same settlement, attestation, and slashing machinery; they differ only in payment timing and cancellation policy.

5.6 Why Markets, Not a Single Centralized Scheduler

The three-market structure is not a UX choice; it is a property required for the chain to function as the open AI protocol. A single centralized scheduler — even a benevolent one — would be a trust assumption equivalent to an OpenAI or an Anthropic. Markets, by contrast, expose the price function explicitly, let any operator participate, and let any caller verify the result. The chain mediates, it does not adjudicate.

6 Agent Registry

6.1 Agents as First-Class Citizens

A Hanzo Agent (`~/work/hanzo/agents`) is an autonomous program that calls models, holds state, and takes actions on behalf of a principal. Without an on-chain registry, agents are indistinguishable from any other API caller and cannot accumulate durable identity, capability, or reputation. The AI Chain promotes agents to first-class citizens with three primitives: *persistent identity*, *capability declaration*, and *on-chain reputation*.

6.2 Persistent Identity

An agent is created on chain with a record:

Field	Definition
<code>agent_id</code>	Hash of (creator address, nonce). Stable for the agent’s lifetime.
<code>controller</code>	Address authorized to update the agent record (or rotate keys, retire the agent).
<code>principal</code>	Address on whose behalf the agent acts. May equal controller.
<code>model_set</code>	Set of allowed HMM <code>model_roots</code> the agent may invoke.
<code>capability_root</code>	Merkle root of the agent’s capability declaration (see below).
<code>spend_cap</code>	Per-epoch ceiling on AI the agent may spend on compute.
<code>reputation</code>	A monotonically updated score in $[0, 1]$ (§6.4).

Table 5: Agent registry record.

The agent’s identity is its `agent_id`. It does not require an externally owned account in the EVM sense; it is referenced as a chain object addressable from any contract through the `AGENT_AUTH` precompile (§2).

6.3 Capability Declaration

A capability declaration is a typed list of actions the agent is authorized to take. Declarations are content-addressed (committed to via `capability_root`) and machine-readable. Examples:

- `infer:hmm:zen-text@*`: invoke any version of zen-text;
- `spend:ai:max=10`: may spend up to 10 AI per epoch;
- `cross:f-chain:fhe-infer:hmm:medic-7b`: may issue confidential inference calls to F-Chain on the named model;

- `settle:c-chain:erc20:USDC@<addr>`: may transfer USDC on the EVM C-Chain.

The declaration is the contract between the agent’s principal and the chain; precompiles enforce it at every action. An agent attempting to exceed its declaration is rejected at the precompile boundary, before any external side effects.

6.4 Reputation

Reputation is a function of the agent’s compute and settlement history:

$$\text{rep}(a) = \sigma(w_1 \cdot \text{success_rate}(a) + w_2 \cdot \log(1 + \text{volume}(a)) + w_3 \cdot \text{age}(a) - w_4 \cdot \text{disputes}(a)) \quad (3)$$

with the logistic σ mapping to $[0, 1]$. Weights w_i are governance parameters. **success_rate** is the fraction of jobs that produced valid TEE-attested receipts; **volume** is total AI spent; **age** is epochs since registration; **disputes** is the count of dispute resolutions against the agent.

Reputation is read by markets and by other agents: an inference operator may price differently for high- vs. low-reputation callers; an agent network may decline to delegate work to a low-reputation peer. The chain publishes reputation; consumers act on it.

6.5 Cross-Chain Authorization

Agents transact across the Lux multi-chain layout (§7) through capability-bound precompiles. To call the EVM C-Chain on the agent’s behalf, a contract invokes **AGENT_AUTH** with (agent_id, target_chain, action). The precompile verifies that the agent’s capability declaration permits the action, decrements **spend_cap**, and emits a B-Chain bridge message. The cross-chain side then sees a single canonical authorization rooted at the agent record on AI Chain.

6.6 Privacy of Agent Memory

The agent registry does not record agent memory or working state on chain; only commitments. Agent memory lives in the agent’s private storage (capsule, in Hanzo Cloud parlance), encrypted under keys custodied by M-Chain MPC if the agent so configures. The chain knows which agent acted; it does not know what the agent thought.

6.7 Worked Example: A Research Agent

A user creates a research agent with the following capability declaration:

```

1 infer:hmm:zen-text@*
2 infer:hmm:zen-vision@*
3 spend:ai:max=50
4 settle:c-chain:none
5 cross:f-chain:none

```

The agent is registered with **spend_cap** 50 AI/epoch and **model_set** = {zen-text, zen-vision}. Over the next epoch the agent runs 1,200 inference calls on zen-text and 240 calls on zen-vision, spending 38.7 AI. Each call produces a TEE-attested receipt referenced by the agent’s record. After 30 successful epochs and zero disputes, the agent’s reputation has saturated near 1.0 and downstream markets begin offering it preferred pricing on reserved capacity.

7 Cross-Chain Integration

The AI Chain is one chain in the Lux multi-chain layout (LP-134, the P/Q/Z/A/B/M/F pipeline). Each peer chain has a single, well-defined purpose; AI Chain composes with them through small, typed precompiles rather than redefining their work.

7.1 C-Chain (EVM)

The Lux C-Chain runs the general-purpose EVM. The AI Chain runs the AIVM, which is a strict superset of the EVM (§2). Any solidity contract that runs on C-Chain runs on AI Chain unmodified; contracts on AI Chain may additionally call the AIVM precompiles.

The two chains exchange value and messages through B-Chain bridges. A C-Chain ERC-20 contract holding USDC can settle the receivable side of a Hanzo inference contract that lives on AI Chain; the bridge message is what makes the cross-chain transfer atomic with respect to the AI receipt.

7.2 F-Chain (FHE Confidential Inference)

F-Chain is the Lux fully homomorphic encryption chain (LP-013) and the home of confidential ERC-20 patterns (LP-067). For inference workloads where the input data must remain encrypted end-to-end, the AI Chain delegates execution to F-Chain through the `F_CALL` precompile.

The flow:

1. Caller encrypts input under CKKS to the operator’s public key on F-Chain.
2. Caller invokes `F_CALL(model_root, fhe_input, max_payment)` on AI Chain.
3. AI Chain emits a B-Chain bridge message routing the FHE ciphertext and the model commitment to F-Chain.
4. F-Chain operator runs the homomorphic forward pass under TEE, emits an encrypted output and a TEE quote.
5. Bridge returns `(encrypted_output, tee_quote)` to AI Chain; the receipt is recorded.
6. Caller decrypts the output under their secret key.

The provider sees only ciphertext throughout. The receipt on AI Chain proves which model produced the result without revealing inputs or outputs. This is the same primitive Hanzo AI Cloud describes for confidential inference (`~/work/hanzo/papers/hanzo-ai-cloud`), now anchored on chain.

7.3 M-Chain (Threshold Custody)

M-Chain is the Lux MPC chain. Model owners with high-value or regulated weights may custody their model decryption keys on M-Chain under threshold MPC: a quorum of custodians must sign to release a decryption shard. The AI Chain references the custody record through the `M_CUSTODY` precompile; an operator that wants to load encrypted weights must request and receive the threshold-signed release.

This composes with HMMs in two useful ways. First, a model owner can publish weights publicly under a permissive license while reserving a specialized fine-tune under threshold custody. Second, regulated domains (medical, financial) can satisfy data-residency requirements by anchoring decryption authority to a custody quorum that includes in-jurisdiction parties.

7.4 B-Chain (Bridges)

B-Chain (LP-134 §5) is the chain of bridges. All cross-chain messages in the AI Chain workflows above flow over B-Chain. The bridge is itself QuasarCert-secured: a B-Chain message attesting that “AI Chain block h recorded receipt r ” is signed by the same three-lane consensus that finalized the underlying block. There is no weaker bridge security tier.

7.5 P-Chain, Q-Chain, Z-Chain (Consensus Substrate)

P/Q/Z are the consensus chains that produce the lanes of every QuasarCert (LP-105). The AI Chain is a consumer of those certificates, not a producer of them, in the same way every other Lux chain is. This is why AI Chain inherits Quasar 3.0 security without modification: the lanes are produced once, on the consensus substrate, and reused by every chain that asks for finality.

7.6 Topology Summary

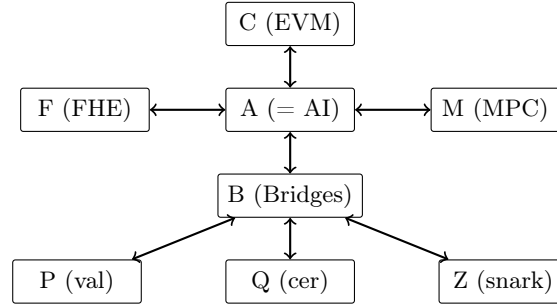


Figure 3: The AI Chain (A) sits at the center: it composes with the EVM (C) above, with bridges (B) below, with FHE (F) on the left, and with MPC (M) on the right. P/Q/Z produce the lanes that finalize every chain.

8 Privacy

8.1 Threat Model

The AI Chain assumes three independent adversaries:

- A *network-level* adversary that observes traffic between callers, operators, and the chain.
- A *provider-level* adversary that operates GPUs and runs inference on caller data.
- A *chain-level* adversary that observes everything written on chain.

Plaintext on chain is public to all three. The privacy primitives below move sensitive material outside the chain entirely or hide it under encryption that no level of the adversary can decrypt without keys.

8.2 Public Inference

The default mode is public inference. The caller sends plaintext input; the operator returns plaintext output; the receipt commits to both. This mode is appropriate for non-sensitive workloads (general chat, public data analysis, code completion on open-source code). The chain records `input_root` and `output_root` commitments — Merkle roots over the plaintext — but the underlying bytes live in CAS and are not stored on chain.

8.3 Confidential Inference (F-Chain Delegation)

For sensitive workloads — medical, financial, defense, and any case where the input itself is the sensitive asset — the AI Chain delegates to F-Chain (§7). The caller encrypts under CKKS to the operator’s public key on F-Chain. F-Chain operators run the homomorphic forward pass under TEE and return ciphertext. Neither the operator nor any chain observer sees plaintext.

The F-Chain inference path inherits the LP-067 confidential ERC-20 pattern in spirit: the on-chain object commits to a value and an authority but does not reveal the value. Here the object is a model query and an output, not a token amount. The TEE quote covers the homomorphic operation by binding (input ciphertext root, model commitment, output ciphertext root) to the GPU’s hardware identity.

8.4 Threshold Custody for Model Weights

Model owners may custody decryption keys for their weights under M-Chain threshold MPC (§7). This addresses two scenarios:

- *Selective release*: the model’s weights are encrypted at rest; an operator that wants to load the model must obtain a threshold-signed decryption shard. Custody quorums can be drawn to satisfy contractual or jurisdictional requirements.
- *Revocable inference*: a model that has been served for a time can be retired by retiring the custody key — operators cannot reload the weights once the custody quorum withholds the shard. Useful for time-bounded research access or for deprecating compromised checkpoints.

8.5 Receipt Privacy

Even with FHE-protected inputs and outputs, the existence of a receipt is observable on chain. The chain records (caller, operator, model_root, block, payment). For workloads where even this metadata is sensitive, callers may use a stealth-address pattern: the caller is a one-shot address, the payment is denominated in a confidential AI balance on F-Chain, and the receipt is encrypted to the caller’s session key. The on-chain footprint becomes a generic “some address paid some operator for some model.”

8.6 Privacy Layering Summary

Mode	Input	Output	Identity
Public	plain	plain	visible
Confidential (F)	FHE	FHE	visible
Custody (M)	plain	plain	visible
Custody + Confidential	FHE	FHE	visible
Confidential + Stealth	FHE	FHE	hidden

Table 6: Privacy modes available on AI Chain. Each row composes independent primitives; the chain validates the receipt regardless of which combination the caller chose.

8.7 What Is Not Hidden

The chain does not hide block height, the validator set, the existence of a receipt, the Quasar lane proofs, or the operator’s stake state. These are network invariants and must be public for the consensus itself to function. Anything tied to the workload (input, output, model weights when custodied, optionally the caller identity) can be hidden behind the primitives above.

9 Security

9.1 Three-Lane Post-Quantum Safety, Inherited

The AI Chain inherits the full Quasar 3.0 security envelope from the Lux consensus substrate (LP-105). Every block is finalized by a QuasarCert that simultaneously requires:

1. A BLS12-381 aggregate signature over the certificate subject;
2. A Ringtail two-round Ring-LWE threshold signature anchored to a Q-Chain ceremony root;
3. A Z-Chain Groth16 SNARK rolling up N ML-DSA-65 validator signatures into a single 192-byte proof plus 32-byte public-input hash.

The certificate subject binds chain id, epoch, round, mode, and the seven chain-specific roots ($P_{\text{val}} \parallel Q_{\text{cer}} \parallel Z_{\text{vk}} \parallel A_{\text{att}} \parallel B_{\text{br}} \parallel M_{\text{mpc}} \parallel F_{\text{fhe}}$) plus the parent block hash and execution roots, making cross-epoch and cross-chain replay structurally impossible. See LP-105 §4 and §6 for the formal QuasarCert definition.

9.2 What an AI Chain Adversary Must Break

To forge a block on the AI Chain, an adversary must *simultaneously*:

- Solve the discrete logarithm on BLS12-381 (P-lane);
- Solve Module-LWE for the Ringtail parameters (Q-lane);
- Solve Module-LWE / MSIS for ML-DSA-65 (Z-lane).

Breaking any single lane, or any two of the three, leaves the third intact and the certificate unforgeable. A future cryptographically relevant quantum computer that breaks BLS does not break the AI Chain; the Q and Z lanes still hold.

9.3 Operator Honesty

Block-level safety guarantees that what the chain accepted, the quorum signed. It does not guarantee that an operator ran the right model on the right input. That second-tier honesty is enforced by TEE attestation:

- Every receipt is accompanied by a vendor-rooted TEE quote (LP-134, A-Chain attestation primitive). The quote covers the loaded model commitment, the runtime image, and the input/output commitments.
- Verifiers re-validate the quote against the vendor root CA on chain. A malformed or unsigned quote rejects the receipt and slashes the operator (Algorithm in §5).
- Periodic spot-check verifiers, drawn by VRF, replay a sampled fraction of jobs in their own TEEs. Discrepancies escalate to full slashing.

Theorem 9.1 (Operator-fraud detection). *Let α be the verifier sampling rate per epoch and ρ the fraction of jobs an attacker chooses to falsify. The probability that a false job goes undetected for k epochs is $(1 - \alpha\rho)^k$. For $\alpha = 0.003$ and $\rho = 0.05$, the probability of evading detection across 1,000 jobs is below 10^{-3} , while the verifier overhead remains below 0.3% of total compute.*

Sketch. Verifier sampling is independent across jobs. The probability of sampling a specific job is α ; the probability of detecting a falsified job conditional on sampling is 1 (the TEE quote replay exposes the divergence by construction). Over k jobs, the unsampled probability is $(1 - \alpha)^{\rho k}$. Substituting: at $\alpha = 0.003$, $\rho = 0.05$, $k = 1000$ gives detection probability > 0.999 . $\square$

9.4 Sybil Resistance

Sybil attacks against the operator side are bounded by stake. A fictitious operator must post $\geq 10,000$ AI of stake; a Sybil farm of N identities must post $10,000 N$ AI. Block reward and inference fees are split per attested job, not per operator identity, so creating identities does not increase share. False quotes from any one identity slash 100% of that identity's stake.

9.5 Liveness

Liveness on the AI Chain requires that the Quasar 3.0 quorum remains intact. LP-105 guarantees safety with $f < n/3$ Byzantine validators; liveness is preserved with $f < n/3$ failed validators provided the remaining $2n/3 + 1$ produce the three lanes. No additional liveness assumption is added on top.

For the markets, liveness requires that operators participate. Idle operators do not earn block reward share, which produces a baseline incentive to attest and serve.

9.6 Replay and Cross-Chain Attacks

Every QuasarCert binds the certificate subject to the chain id, the epoch, and the seven chain-root tuple. The same finalized block on AI Chain cannot be replayed on C-Chain or any other chain in the pipeline because the chain id and the Z-Chain Groth16 verification key differ. Cross-chain bridge messages on B-Chain include a B-Chain block height and a destination chain id; replay across chains is similarly impossible.

9.7 Audit Surface

The AI Chain exposes the following for external audit:

- All HMM registry records and their training history DAG;
- All operator listings, jobs, receipts, and TEE quotes;
- All agent registry records and capability declarations;
- All QuasarCert lane components per block (P, Q, Z);
- All slashing events, with the underlying evidence (malformed quote, divergent replay, lane-incomplete cert).

This is, by design, more transparent than any centralized AI service. The premise of the AI Chain is that the cost of that transparency is paid down by the ability to verify any output without asking the provider.

10 Deployment

10.1 Activation Boundary

The AI Chain activates on **25 December 2025**, the same boundary at which Lux Quasar 3.0 (LP-105), QuasarSTM (LP-010), QuasarGPU (LP-132), and the unified A-Chain attestation surface (LP-134) become canonical on the Lux primary network. The spec described in this paper is frozen as of **15 December 2025**.

There is no separate activation for the AI Chain. Activating Quasar 3.0 activates the AIVM, the HMM registry, the agent registry, the three markets, and the AI block reward.

10.2 Pre-Activation Genesis State

At activation block 0, the chain state is seeded with:

- The 1,000,000,000 AI genesis allocation (§3) distributed to the addresses recorded in the genesis manifest;
- The five core Hanzo model registrations (`hmm:zen-text`, `zen-vision`, `zen-audio`, `zen-code`, `zen-embed`) at version 1.0 (§4);
- The vendor root CA set required to validate TEE quotes (Intel TDX, AMD SEV-SNP, NVIDIA H100/H200/B100 Confidential Computing, ARM CCA);
- The genesis validator set inherited from the Lux primary network at the activation block;
- The initial epoch parameters: 500ms block time, 4 AI block reward, 70/30 base-fee burn split.

10.3 Operator Onboarding

Operators may begin onboarding from the spec freeze date (15 December 2025). Onboarding steps:

1. Acquire AI from genesis distribution (public sale or ecosystem grant);
2. Stake $\geq 10,000$ AI to the operator registry;
3. Provision TEE-capable hardware (any of the supported vendor attestation roots);
4. Load one or more registered HMMs;
5. Post inference, training, or pool listings.

Operator listings posted before activation are queued; matching begins at block 0.

10.4 Caller Onboarding

Callers (humans, applications, agents) need only an AI balance and an EVM-compatible wallet to interact with the AIVM precompiles. The Hanzo LLM Gateway (`~/work/hanzo/llm`) and the Hanzo Agent SDK (`~/work/hanzo/agents`) integrate the chain calls behind their existing OpenAI-compatible APIs, so existing client code requires no changes beyond endpoint and credential swaps.

10.5 Activation Roadmap

10.6 Post-Activation: First 90 Days

- *Operator ramp*: target 200 staked operators by activation + 30 days, 1,000 by + 90 days.
- *Inference TPS*: target sustained 10,000 inference receipts per minute by + 30 days, scaling with operator count.
- *Training bounties*: first community-funded fine-tune bounty by + 14 days; first cleared bounty by + 45 days.
- *Cross-chain receipts*: first F-Chain confidential inference receipt by + 7 days; first M-Chain custody-gated model load by + 21 days.

Date (2025)	Milestone
15 December	Spec freeze. This document final. AIVM client + node binaries tagged. Operator staking opens.
17 December	Genesis manifest published. HMM root commitments anchored. Vendor TEE root CA set pinned.
20 December	Final review window closes. Validator set enumerated.
22 December	Operator dry run on the AI Chain shadow network.
24 December	Validator readiness gate. Quorum confirms LP-010 / LP-105 / LP-132 / LP-134 readiness.
25 December	Activation block. AIVM live, AI block reward live, compute markets open, agent registry open.

Table 7: Activation timeline. All dates 2025; all dates inside or before the LP-105 activation boundary.

10.7 Where Things Run

The AI Chain code lives under `~/work/hanzo/aci` (node binary, AIVM precompiles, registry contracts). Reference clients live under `~/work/hanzo/agents`, `~/work/hanzo/llm`, and the Hanzo Cloud control plane (`~/work/hanzo/cloud`). The chain is operated on the Lux mainnet validator set; AI-specific operators run anywhere they can produce a valid vendor TEE quote.

11 Conclusion

The Hanzo AI Chain is the AI Virtual Machine: A-Chain (LP-134), the unified Lux attestation chain, viewed as the open AI protocol. There is one ledger, one consensus (Quasar 3.0, LP-105), one execution substrate (QuasarGPU, LP-132), and one scheduling model (QuasarSTM, LP-010). On top of that single ledger sit the Hanzo model registry, the agent registry, and three composable compute markets, all denominated in the native AI coin and all secured by three-lane post-quantum certificates.

The economic claim is simple: a content-addressed registry of models, a verifiable receipt for every inference call, a market for compute priced in a coin that burns under load, and an agent layer with durable identity together produce an open AI protocol that does not require a hyperscaler. The technical claim is that this protocol can be delivered without a separate chain and without a weaker security tier: A-Chain’s three-lane envelope and QuasarGPU’s TEE primitives are exactly the primitives an open AI protocol needs, and they are already in production on the Lux primary network.

Activation is 25 December 2025. The spec is frozen as of 15 December 2025. From that boundary forward, AI is the native coin of the Hanzo Network, HMMs are first-class on-chain objects, agents have persistent identity and capability, and any operator with a TEE-attestable GPU can sell compute into a market that any caller can price-discover and any auditor can verify.

Cross-references. This paper is part of the Hanzo papers collection (`~/work/hanzo/papers`). Related work in the same collection: *Hanzo ACI: AI Chain Infrastructure* (`hanzo-aci/`); *Hanzo AI Cloud: Confidential Inference, Agent Hosting, and Model Serving* (`hanzo-ai-cloud/`); *Hanzo Agent SDK* (`hanzo-agent-sdk/`); *Hanzo Cloud Infrastructure* (`hanzo-cloud-infrastructure/`).

Related Lux papers (`~/work/lux/papers`): *LP-010: QuasarSTM* (`lp-010-quasar-stm-3/`); *LP-105: Quasar 3.0 Consensus* (`lp-105-quasar-consensus/`); *Lux A-Chain Attestation* (`lux-achain-attestation/`); *Lux Decentralized Cloud* (`lux-decentralized-cloud/`).

References

- [1] Lux Industries. *LP-010: QuasarSTM 3.0 — Lane-based Software Transactional Memory*. Lux Network Technical Specifications, December 2025.
- [2] Z. Kelling. *LP-105: Quasar 3.0 Consensus — Three-Lane Post-Quantum Certificates*. Lux Network Technical Specifications, December 2025.
- [3] Lux Industries. *LP-132: QuasarGPU — GPU-Resident Verification and Execution Substrate*. Lux Network Technical Specifications, December 2025.
- [4] Lux Industries. *LP-134: A-Chain — Unified Attestation for the P/Q/Z/A/B/M/F Pipeline*. Lux Network Technical Specifications, December 2025.
- [5] Lux Industries. *LP-013: F-Chain — Fully Homomorphic Encryption Substrate*. Lux Network Technical Specifications, 2025.
- [6] Lux Industries. *LP-067: Confidential ERC-20 on F-Chain*. Lux Network Technical Specifications, 2025.
- [7] Lux Industries. *LP-136: Lux Decentralized Cloud — Compute Settlement Layer*. Lux Network Technical Specifications, 2025.
- [8] Hanzo AI Research. *Hanzo ACI: AI Chain Infrastructure for Decentralized Compute Markets*. Hanzo Papers, 2025.
- [9] Z. Kelling. *Hanzo AI Cloud: Confidential Inference, Agent Hosting, and Model Serving*. Hanzo Papers, 2025.
- [10] A. Worring and Z. Kelling. *Hanzo Agent SDK: Building AI Agents with Orchestration, Tool Use, and Memory Systems*. Hanzo Papers, 2025.